

Context Engineering Tools & Patterns Cheatsheet

Hands-On Context Engineering — O'Reilly Live Training · Lucas Soares

CAUTION

Context engineering is the discipline of designing the architecture that feeds an LLM the **right information** at the **right time**. It transforms high-entropy human intentions into low-entropy representations machines can process. Unlike humans, machines cannot "fill in the gaps."

The Four Context Operations (W/S/C/I)

Every context engineering strategy maps to one of these four fundamental operations.

- **WRITE** — Create context that doesn't exist yet. Plans, instructions, scratchpads, learned knowledge, memories. Agent writes files for future reference.
- **SELECT** — Choose the most relevant context from available sources. Semantic search, structural search (grep/glob), adaptive retrieval, dynamic tool loading.
- **COMPRESS** — Reduce size while preserving essential information. Summarization, pruning, offloading to disk, spawning subagents for token-heavy ops.
- **ISOLATE** — Separate context types to prevent interference. Subagent isolation, dynamic tool loading, working memory scratchpads, state objects.

The Four Context Failure Modes

Drew Breunig's taxonomy — each mode has distinct symptoms and specific fix levers.

- **POISONING** — Hallucination enters context and compounds. Agent treats its own prior output as ground truth. *Signal: bad fact reappears across steps*. Fix: **Isolate + Select**
- **DISTRACTION** — Too much history causes the agent to repeat past behavior instead of reasoning fresh. *Signal: agent repeats prior actions*. Fix: **Compress + Select**
- **CONFUSION** — Irrelevant tools or documents crowd the context, leading to wrong tool selection. *Signal: irrelevant tools/docs selected*. Fix: **Select + Isolate**
- **CLASH** — Contradictory information creates conflicting assumptions and unreliable output. *Signal: flip-flopping between approaches*. Fix: **Isolate + Select**

Quick Reference: Failures to Fixes

- **Poisoning** → Primary: Isolate + Select | Supporting: Write (claims + sources) | Prune hallucinated turns, add validation instructions
- **Distraction** → Primary: Compress + Select | Supporting: Write (persistent plan) | Summarize stale history, select smaller working set
- **Confusion** → Primary: Select + Isolate | Supporting: Tool Loadout (dynamic) | Load only relevant tools per step, isolate nice-to-have context
- **Clash** → Primary: Isolate + Select | Supporting: Compress (resolved summary) | Quarantine tasks into separate contexts, set precedence rules

The Five-Step Debugging Loop

1. **Inspect Trace** — Walk through messages, tool calls, and outputs turn by turn
2. **Tag Failure** — Which mode? Poisoning / Distraction / Confusion / Clash
3. **Pick a Lever** — Use the Write / Select / Compress / Isolate framework
4. **Re-run & Evaluate** — Track token usage per step AND measure outcome quality
5. **Iterate** — Context engineering is iterative, not one-shot

ALERT

Don't just optimize for fewer tokens — **measure outcome quality**. A fix that reduces tokens but degrades results is worse.

Context Budget Problem

Every agent must make critical decisions about its finite context budget:

- **Keep Active** — What stays in the context window right now?
- **Store Externally** — What goes to disk or DB for later?
- **Compress** — What can be summarized to save space?
- **Reserve** — How much capacity for reasoning?

CAUTION

Bigger windows are NOT better. Models now support 100K, 200K, even 1M+ tokens — but performance degrades well before hitting limits. Gemini 2.5 Pro decreased performance beyond 100K tokens. Llama 3.1 405B hit distraction ceiling at ~32K tokens.

Context Prioritization Hierarchy

- **CRITICAL** (always include): Current code/content being modified, specific error messages, expected output format, active task requirements
- **HELPFUL** (include if space permits): Project structure, dependencies in use, coding conventions, related documentation
- **OPTIONAL** (include only if needed): Historical decisions, future plans, tangential examples, background reference

Production Patterns from Real Systems

The Static-to-Dynamic Spectrum

- **Static** — Always loaded (CLAUDE.md, .cursorrules). Predictable but stale, always-costing tokens.
- **Tiered** — Hot/warm/cold layers loaded by role or trigger. Structured, requires infrastructure.
- **Dynamic** — Agent discovers and fetches on demand (Cursor, Claude Code grep/glob). Token-efficient, always fresh.
- **Self-Improving** — Contexts evolve based on execution feedback (ACE framework). No manual maintenance.

Manus's 5 Production Strategies

1. **Context Compaction & Summarization** — Reversible compaction (store file paths, not contents) + lossy summarization triggered at ~128K tokens
2. **Communication Over Shared Context** — Fresh sub-agents for discrete tasks instead of sharing full conversation history
3. **Hierarchical Action Spaces** — Limit visible tools to ~20 atomic tools. Level 2: sandbox utilities. Level 3: code libraries
4. **Agent-as-Tool Pattern** — Sub-agents as deterministic functions with defined inputs, outputs, and JSON schemas
5. **Implementation Best Practices** — Avoid dynamic RAG-based tool retrieval (breaks KV cache). Set pre-rot thresholds at 70-80% utilization

Static Context Files: The ETH Zurich Study

- **-3%** success rate: LLM-generated context files actually hurt performance
- **+4%** success rate: Human-written context files give marginal improvement
- **+20%** cost increase across both file types (14-22% more reasoning tokens)
- **Best practice:** Keep it minimal (10-20 lines), specific tooling only, don't duplicate README content, review quarterly

Key Retrieval Approaches

- **Vector Store (RAG)** — Classic chunking + embeddings + similarity search. Requires indexing, chunking decisions.
- **Agentic Search** — llm.txt file with URLs + descriptions. Agent fetches docs on demand. Extremely effective with good descriptions.
- **Context Stuffing** — Feed all docs directly to the agent. Brute force. Expensive and degrades quality.

- **Progressive Disclosure** — Start small, expand as needed. Don't pre-load everything. File paths and links — load full content only when needed.

The Compression Toolkit

- **Prompted Summarization** — Compress with high recall. Exhaustive bullet points so the agent knows whether to retrieve full context.
- **Auto-Compaction** — Automatic compression nearing limits. Claude Code auto-compacts at 95% capacity.
- **Trimming** — Selective removal using heuristics. Keep only recent N messages or LLM-scored relevance.
- **Prompt Caching** — Avoid re-sending invariant context. System prompts, tool definitions — cache, don't reprocess.
- **Context Offloading** — Write full results to disk, return summary + pointer to the agent. Agent fetches on demand.

ALERT

Risk: Summarization loses information. Always **offload to disk before summarizing** to retain raw context as a safety net.

Course Demos Quick Reference

Demo 1: Context Engineering Principles (Claude Code)

Session 1 — Live CLI demos with Claude Code. No code to run — uses `claude` directly.

- `/context` — Make the context window visible (see what the model sees)
- `/cost` — Track token consumption per session
- `/clear` — Reset context to demonstrate context rot recovery
- `Read tool` — Ground the model in documents (beats parametric memory)
- `CLAUDE.md` — Static context file as persistent instructions
- Lost in the Middle: Same question, same document — answer quality depends on WHERE the fact is positioned

Demo 2: Agentic Retrieval (Interactive TUI)

Session 2 — Run: `uv run app.py` in `demos/agentic-retrieval/`

- Hand-rolled agent loop — `self.messages` IS the context window, made explicit and inspectable
- Progressive disclosure retrieval: `list_documents` (cheap) → `search_documents` (moderate) → `get_document` (expensive)
- System prompt ~200 tokens (WRITE lever), tool definitions ~800 tokens fixed overhead per call
- Slash commands: `/context` shows the live messages array, `/stats` shows token counts
- TF-IDF search with `scikit-learn` — no vector DB needed for small knowledge bases
- Teaching point: every `append()` to `messages` is a permanent token cost

Demo 3: Context Failures (Jupyter Notebook)

Session 3 — Open: `context_failures.ipynb` in `demos/context-failures/`

- **Poisoning**: Hallucinated "AI Code Review" feature compounds across turns. Fix: prune + validation instruction.
- **Distraction**: 5 verbose brute-force turns bias the model away from optimal DP solution. Fix: summarize to one-liner.
- **Confusion**: 12 irrelevant tool schemas (~2,400 tokens overhead) for a simple factual question. Fix: remove all tools.
- **Clash**: Budget trip plan contradicts later luxury requirements. Fix: prune old plan + add CURRENT REQUIREMENTS override.
- Each scenario shows broken vs. fixed with real token cost comparison and HTML dashboard

Demo 4: Chat with Artifacts (Web App)

Session 4 — Run: `uv run app.py` in `demos/chat-with-artifacts/` → open `http://127.0.0.1:8000`

- Layered system prompt: Layer 1 (fixed persona ~150 tok) + Layer 2 (static artifact schemas ~400 tok) + Layer 3 (dynamic state 0-200 tok)
- 7 artifact types: `selectable_options`, `flashcard_deck`, `checklist`, `flowchart`, `semantic_zoom`, `inline_quiz`, `concept_explorer`
- Structured output via `tool_choice` — guaranteed JSON matching schema. "The prompt provides the content; the schema provides the shape."
- `ArtifactRegistry` injects dynamic context: tracks what's been created, tells model to build on existing artifacts
- `structured_outputs_demo.py` — Same prompt, 3 different schemas, 3 different output shapes. Same input cost.

Key Numbers to Remember

- **500K tokens** per run for naive deep research agents | **\$1-2** cost per single agent run
- **~50-100** tool calls in production agents | **83.9%** of context consumed by tool outputs
- **n^2 attention complexity** — more tokens = exponentially more pairwise relationships to score
- **95% capacity** — Claude Code auto-compaction trigger | **70-80%** — Manus pre-rot threshold
- **46.9%** token reduction from Cursor's dynamic context discovery | **44%** performance improvement from dynamic tool loading
- **~150-200 instructions** — frontier LLM instruction-following limit. Claude Code's system prompt already uses ~50.

Core Principles

INFORMATION

"Find the smallest set of high-signal tokens that maximize the likelihood of your desired outcome." — Anthropic, Effective Context Engineering for AI Agents

1. **Context engineering is entropy reduction** — High-entropy intentions into low-entropy representations machines understand
2. **Systems, not prompts** — The full architecture of information flow: storage, management, and usage
3. **Four operations: Write, Select, Compress, Isolate** — Every strategy maps to one of these
4. **Quality over quantity** — Bigger windows don't help. Focused, relevant, well-managed context does.
5. **Context hygiene is continuous** — Monitor, prune, refresh, and validate throughout the lifecycle
6. **Agentic retrieval beats naive RAG** — Simple tools (grep/glob) with progressive disclosure outperform complex pipelines
7. **Read tasks parallelize, write tasks don't** — Sub-agents for research, single-threaded execution for code generation
8. **Offload before you summarize** — Save raw context to disk first, then compress. Summarization is lossy.

Hands-On Context Engineering — O'Reilly Live Training · Lucas Soares · 2026